ESTRUCTURAS DE DATOS · GIISI

Búsqueda y ordenación

Apuntes resumidos: conceptos generales, tablas, búsqueda lineal y binaria, los ocho métodos de ordenación interna, y los ficheros con su ordenación y búsqueda externa. Con ejemplos paso a paso y el pseudocódigo de cada algoritmo.



Índice

1. Introducción

Los 4 módulos del temario

2. Conceptos generales

El puntero
El valor NULO
Reservar y liberar memoria

3. Tablas (arrays)

Vocabulario
Definir una tabla
Dar valor a las componentes
Operaciones básicas

4. Algoritmos de búsqueda

Búsqueda lineal (secuencial)
Búsqueda binaria (dicotómica)

5. Algoritmos de ordenación

6. Inserción directa

7. Inserción binaria

8. Selección directa

9. Intercambio directo

Mejora: burbuja con test

10. Sacudida

Sacudida con test
Sacudida con salto

11. Shell

12. QuickSort

13. MergeSort

La operación de mezcla

14. Ficheros secuenciales

Fichero, registro, campo y clave
Operaciones básicas

15. Ordenación externa

16. Mezcla directa

17. Mezcla natural

18. Clasificación de raíz

19. Búsqueda en ficheros

20. Resumen y comparativa

Ordenación externa: comparativa
¿De qué depende el número de iteraciones?
Trampas típicas de examen

21. Tabla de métodos de ordenación

Qué hace cada método y dónde se puede usar
¿Cuál es más rápido?
En resumen

TEMA 0

Introducción

Esta herramienta presenta las **estructuras de datos fundamentales** y los **algoritmos** que las manipulan. Los algoritmos se describen en **pseudocódigo** y se acompañan de representaciones gráficas.

Los 4 módulos del temario

1. **Presentación de la herramienta**
2. **Búsqueda y ordenación**
   en estructuras internas y externas
3. **Estructuras lineales**
4. **Estructuras no lineales**

Estos apuntes cubren los **conceptos generales** (memoria dinámica), las **tablas**, la **búsqueda y ordenación interna** (en vectores) y la **búsqueda y ordenación externa** (en ficheros).

TEMA 1

Conceptos generales

Las estructuras de datos se representan de dos formas básicas: con **vectores** o con **memoria dinámica**. Para la segunda hacen falta los punteros.

El puntero

> **Puntero:** variable que almacena una **dirección de memoria**. Se dice que "apunta" a esa zona de memoria.

Al declararlo se indica su **nombre** y su **tipo asociado**, que determina a qué tipo de elementos podrá apuntar:

```
P puntero a enteros
```

P puede apuntar tanto a **variables estáticas** como a **variables dinámicas** de ese tipo.

Variable estática

Su memoria se solicita en **tiempo de compilación**. Tiene nombre.

Variable dinámica

Su memoria se solicita en **tiempo de ejecución**. **No tiene nombre**: solo se accede a ella mediante un puntero que apunte a su zona de memoria.

El valor NULO

Valor especial que se asigna a un puntero para indicar que **no apunta a ninguna dirección de memoria válida**.

```
P ← NULO
```

Reservar y liberar memoria

1. **Reservar.** `RESERVAR_MEMORIA` recibe el **tamaño del bloque** que se pide y devuelve la **dirección de comienzo** del bloque asignado, o **NULO** si la solicitud no tuvo éxito. El tamaño lo calcula la función `tamaño(T)`, que da el espacio que necesita un dato del tipo T.
2. **Comprobar.** Antes de usar la memoria conviene comprobar que la reserva funcionó: operar sobre memoria no asignada provoca un error en el programa.
3. **Liberar.** Cuando la variable dinámica ya no hace falta, `LIBERAR(P)` devuelve al sistema la memoria a la que apunta P.

```
P ← RESERVAR_MEMORIA(tamaño(entero))

Si (P = NULO) entonces
   ESCRIBIR "error en la reserva de memoria"
   ACABAR   ** u otra operación
fin-si

LIBERAR(P)
```

> 💡 **Idea clave:** reservar → comprobar si es NULO → usar → liberar.

Tablas (arrays)

> **Tabla o array:** número **fijo, finito y ordenado** de elementos, **todos del mismo tipo** y bajo un **nombre común**.

Vocabulario

TÉRMINO	SIGNIFICADO
Componentes	Los elementos de la tabla (cada constante o variable).
Nombre	Identificador que hace referencia global a todos los elementos.
Índice	Indica la posición relativa de una componente. Puede ser un **número entero**, una **variable numérica entera** o una **expresión entera**.
Dimensión	**Número de índices** necesarios para identificar una componente.
Tamaño	**Número de componentes**. Con varias dimensiones hay un tamaño por dimensión.

1 dimensión tabla unidimensional o **vector**	**2 dimensiones** tabla bidimensional o **matriz**	**3 o más** tabla multidimensional o **poliedro**

Según el lenguaje, el primer elemento de cada dimensión es el **0** o el **1**. En los apuntes los índices empiezan en 1.

Definir una tabla

Hay que indicar el **nombre**, el **tamaño** (entre paréntesis o corchetes, separados por comas si hay varias dimensiones) y el **tipo** de los elementos. La definición reserva memoria para todos los elementos.

```
nombre_vector(n) de tipo T
nombre_matriz(n, m) de tipo T
```

> 💡 **Número de elementos** = producto de los tamaños de todas las dimensiones. Ejemplo: `M(4, 5)` tiene 4 × 5 = 20 elementos.

Dar valor a las componentes

Al crear la tabla, sus componentes tienen **valor indeterminado**, salvo que el lenguaje dé un valor por defecto a cada tipo.

A **Inicialización:** dar valor a todas las componentes **en el momento de declararla**.

```
V(3) de enteros ← (100, 20, 35)
```

índice	1	2	3
V	100	20	35

B **Asignación individual:**

```
V(1) ← 48
```

V	48	20	35

C **Lectura:** `LEER V(I)`

Operaciones básicas

Búsqueda

Determinar si un valor aparece en la tabla. El resultado puede indicar si hubo éxito, si aparece varias veces, en qué posición o posiciones...

Ordenación

Reorganizar los elementos según un criterio, de forma **ascendente** o **descendente**.

TEMA 2 · BÚSQUEDA

Algoritmos de búsqueda

Se busca un valor **X** en un vector **V** de tamaño **N** y tipo T, que **puede tener valores repetidos**.

Si el vector no está lleno, los módulos reciben como segundo parámetro el **número de elementos realmente almacenados**. Se supone que estos ocupan **posiciones consecutivas desde la primera**.

Búsqueda lineal (secuencial)

Vectores desordenados | Vectores ordenados

Se recorre el vector **de un extremo al opuesto** hasta encontrar una componente igual a X o hasta haber recorrido todo el vector, lo que ocurra primero. En el segundo caso se indica que X no está.

- Si X **aparece varias veces**, se encuentra el que tenga el índice **más próximo al extremo donde empieza el recorrido**.
- Si el vector está **ordenado ascendentemente**, la búsqueda puede terminar al encontrar un valor **mayor que X**: X ya no puede aparecer. Se sale del bucle al encontrar un valor ≥ X o al llegar al último elemento, y hay éxito si el valor donde se detuvo es **igual a X**.

EJEMPLO Ejemplo: buscar X = 14 en (15, 18, 7, 75, 14, 13, 43, 40)

índice	1	2	3	4	5	6	7	8	
I = 1	15	18	7	75	14	13	43	40	V(1) = 15 ≠ X → I ← I+1
I = 2	15	18	7	75	14	13	43	40	V(2) = 18 ≠ X → I ← I+1
I = 3	15	18	7	75	14	13	43	40	V(3) = 7 ≠ X → I ← I+1
I = 4	15	18	7	75	14	13	43	40	V(4) = 75 ≠ X → I ← I+1
I = 5	15	18	7	75	14	13	43	40	V(5) = 14 = X → encontrado

comparado · encontrado · ya descartado

{ } **Pseudocódigo · vector que puede estar desordenado**

```
MÓDULO busqueda_lineal
DATOS
   PARÁMETROS
      Recibe V() de tipo T
      Recibe N entera
      Recibe X de tipo T
   VARIABLES
      I entera
INICIO
   I ← 1
   Mientras ( (V(I) ≠ X) y (I < N) ) hacer
      I ← I+1
   fin-mientras

   Si ( V(I) = X ) entonces
      Escribir "Encontrado en posición", I
   sino
      Escribir "No encontrado"
   fin-si
FIN
```

{ } **Pseudocódigo · vector ordenado**

```
MÓDULO busqueda_lineal_ordenado
DATOS
   PARÁMETROS
      Recibe V() de tipo T
      Recibe N entera
      Recibe X de tipo T
   VARIABLES
      I entera
INICIO
   I ← 1
   Mientras ( (V(I) < X) y (I < N) ) hacer
      I ← I+1
   fin-mientras

   Si ( V(I) = X ) entonces
      Escribir "Encontrado en posición", I
   sino
      Escribir "No encontrado"
   fin-si
FIN
```

La única diferencia entre las dos versiones está en la condición del bucle: `V(I) ≠ X` en la general y `V(I) < X` en la de vector ordenado.

Búsqueda binaria (dicotómica)

Solo vectores ordenados

Con los datos en orden creciente, se compara X con la **componente central**:

¿V(CEN) = X?	¿V(CEN) > X?	¿V(CEN) < X?
encontrado	seguir en la mitad **izquierda** `DER ← CEN-1`	seguir en la mitad **derecha** `IZQ ← CEN+1`

Se repite sobre la mitad elegida hasta encontrar el valor o hasta que el **intervalo de búsqueda se anule**. En cada paso el intervalo se reduce aproximadamente a la mitad.

💡 Si X está repetido, se obtiene **uno** de ellos (los repetidos son consecutivos), no necesariamente el primero.

EJEMPLO Ejemplo con éxito: buscar 40

índice	1	2	3	4	5	6	7	8	9	
IZQ=1 DER=9 CEN=5	7	13	14	15	18	26	40	43	51	18 < 40 → IZQ ← 6
IZQ=6 DER=9 CEN=7	7	13	14	15	18	26	40	43	51	V(7) = 40 → encontrado

elemento central · encontrado · fin sin éxito · fuera del intervalo

EJEMPLO Ejemplo sin éxito: buscar 20

índice	1	2	3	4	5	6	7	8	9	
IZQ=1 DER=9 CEN=5	7	13	14	15	18	26	40	43	51	18 < 20 → IZQ ← 6
IZQ=6 DER=9 CEN=7	7	13	14	15	18	26	40	43	51	40 > 20 → DER ← 6
IZQ=6 DER=6 CEN=6	7	13	14	15	18	26	40	43	51	V(6) = 26 ≠ 20 e IZQ ≥ DER → no existe

elemento central · encontrado · fin sin éxito · fuera del intervalo

Pseudocódigo

```
MÓDULO busqueda_binaria
DATOS
   PARÁMETROS
      Recibe V() de tipo T
      Recibe N entera
      Recibe X de tipo T
   VARIABLES
      IZQ, DER, CEN enteras
INICIO
   IZQ ← 1
   DER ← N
   CEN ← (N+1)\2          ** división entera

   Mientras ( (V(CEN) ≠ X) y (IZQ < DER) ) hacer
      Si ( V(CEN) > X ) entonces
         DER ← CEN-1
      sino
         IZQ ← CEN+1
      fin-si
      CEN ← (IZQ+DER)\2
   fin-mientras

   Si ( V(CEN) = X ) entonces
      Escribir "encontrado en la posición", CEN
   sino
      Escribir "no existe el valor buscado"
   fin-si
FIN
```

TEMA 2 · ORDENACIÓN

Algoritmos de ordenación

Ascendente (creciente)

Valores **menores al principio** y mayores al final.

Descendente (decreciente)

Valores **mayores al principio** y menores al final.

- Los valores **repetidos** quedan en **posiciones consecutivas**.
- Los valores **alfanuméricos** se ordenan por **orden lexicográfico**.
- Si cada componente guarda **datos compuestos** (nombre, edad, DNI...), se ordena por **alguno de esos datos**.

En todos los métodos se ordenan los N elementos de un vector V de forma **ascendente**. Todos los ejemplos usan el mismo vector: **(15, 18, 7, 75, 14, 13, 43, 40)**.

Para ordenar de forma **descendente** basta con invertir las comparaciones. Por ejemplo, en la burbuja se intercambia si `V(I) < V(I+1)` en lugar de `V(I) > V(I+1)`.

ORDENACIÓN · 1

Inserción directa

También llamado **método de la baraja**

Se toma cada elemento y se coloca en su sitio respecto a los que tiene a su **izquierda**, que ya están ordenados entre sí, igual que al ordenar las cartas de una mano.

Para cada elemento **desde el segundo hasta el último**:

1. Se copia el valor actual en una variable auxiliar (`AUX`).
2. Desde el elemento anterior hasta el primero, se desplazan **un lugar a la derecha** todos los que sean **mayores** que AUX, para dejar un hueco.
3. Se coloca AUX en el hueco.

índice	1	2	3	4	5	6	7	8	
inicio	15	18	7	75	14	13	43	40	
I = 2	15	18	7	75	14	13	43	40	AUX = 18: desplaza 0 → V(2)
I = 3	7	15	18	75	14	13	43	40	AUX = 7: desplaza 2 → V(1)
I = 4	7	15	18	75	14	13	43	40	AUX = 75: desplaza 0 → V(4)
I = 5	7	14	15	18	75	13	43	40	AUX = 14: desplaza 3 → V(2)
I = 6	7	13	14	15	18	75	43	40	AUX = 13: desplaza 4 → V(2)
I = 7	7	13	14	15	18	43	75	40	AUX = 43: desplaza 1 → V(6)
I = 8	7	13	14	15	18	40	43	75	AUX = 40: desplaza 2 → V(6)
fin	7	13	14	15	18	40	43	75	ordenado

elemento insertado · zona ordenada (aún no definitiva) · ordenado

⚠️ **Ojo en el examen:** tras cada iteración, la parte izquierda está **ordenada entre sí**, pero sus elementos **no están en su posición definitiva**: un valor pequeño que llegue después puede desplazarlos. En el ejemplo, el 7 entra en la iteración 3 y mueve al 15 y al 18.

Pseudocódigo

```
MÓDULO ordenacion_insercion_directa
DATOS
   PARÁMETROS
      Transforma V() de tipo T
      Recibe N entera
   VARIABLES
      I, J enteras
      AUX de tipo T
INICIO
   Para I desde 2 hasta N
      AUX ← V(I)
      J ← I
      Mientras ( (V(J-1) > AUX) y (J > 1) ) hacer
         V(J) ← V(J-1)
         J ← J-1
      fin-mientras
      V(J) ← AUX
   fin-para
FIN
```

ORDENACIÓN · 2

Inserción binaria

Variante de la inserción directa: la búsqueda secuencial del hueco se cambia por una **búsqueda binaria**. Se puede usar porque la zona donde se busca ya está **ordenada**.

Para cada elemento de la posición I (de la 2 a la N):

1. **Búsqueda binaria** en las posiciones **[1, I−1]** para saber qué posición debe ocupar.
2. Los elementos de ese intervalo **mayores** que él se desplazan una posición a la derecha.
3. Se coloca el valor en su nueva posición.

índice	1	2	3	4	5	6	7	8	
inicio	15	18	7	75	14	13	43	40	
I = 2	15	18	7	75	14	13	43	40	binaria en [1,1] → IZQ = 2
I = 3	7	15	18	75	14	13	43	40	binaria en [1,2] → IZQ = 1
I = 4	7	15	18	75	14	13	43	40	binaria en [1,3] → IZQ = 4
I = 5	7	14	15	18	75	13	43	40	binaria en [1,4] → IZQ = 2
I = 6	7	13	14	15	18	75	43	40	binaria en [1,5] → IZQ = 2
I = 7	7	13	14	15	18	43	75	40	binaria en [1,6] → IZQ = 6
I = 8	7	13	14	15	18	40	43	75	binaria en [1,7] → IZQ = 6

posición hallada con búsqueda binaria zona ordenada

⚠️ **Ojo:** la búsqueda binaria solo sirve para **encontrar la posición**. Los **desplazamientos siguen siendo necesarios**. Tras cada iteración el vector queda igual que con la inserción directa.

Pseudocódigo

```
MÓDULO ordenacion_insercion_binaria
DATOS
   PARÁMETROS
      Transforma V() de tipo T
      Recibe N entera
   VARIABLES
      I, J enteras
      IZQ, DER, CEN enteras
      AUX de tipo T
INICIO
   Para I desde 2 hasta N
      AUX ← V(I)
      IZQ ← 1
      DER ← I-1
      Mientras ( IZQ ≤ DER ) hacer
         CEN ← (IZQ+DER)\2      ** división entera
         Si ( V(CEN) ≤ AUX ) entonces
            IZQ ← CEN+1
         sino
            DER ← CEN-1
         fin-si
      fin-mientras

      J ← I
      Mientras ( J > IZQ ) hacer
         V(J) ← V(J-1)
         J ← J-1
      fin-mientras
      V(IZQ) ← AUX
   fin-para
FIN
```

ORDENACIÓN · 3

Selección directa

Busca una y otra vez el **menor** de los elementos que quedan por ordenar y lo pone en su **posición definitiva**.

En cada iteración I (hay **N−1** en total):

1. Se selecciona el **menor** valor entre la posición I y la última.
2. Se **intercambia** con el de la posición I.

índice	1	2	3	4	5	6	7	8	
inicio	15	18	7	75	14	13	43	40	
I = 1	7	18	15	75	14	13	43	40	menor de V(1..8) = 7 (pos. 3) ↔ V(1)
I = 2	7	13	15	75	14	18	43	40	menor de V(2..8) = 13 (pos. 6) ↔ V(2)
I = 3	7	13	14	75	15	18	43	40	menor de V(3..8) = 14 (pos. 5) ↔ V(3)
I = 4	7	13	14	15	75	18	43	40	menor de V(4..8) = 15 (pos. 5) ↔ V(4)
I = 5	7	13	14	15	18	75	43	40	menor de V(5..8) = 18 (pos. 6) ↔ V(5)
I = 6	7	13	14	15	18	40	43	75	menor de V(6..8) = 40 (pos. 8) ↔ V(6)
I = 7	7	13	14	15	18	40	43	75	menor de V(7..8) = 43 (pos. 7) ↔ V(7)

posición definitiva posición intercambiada

💡 Tras la iteración I, los elementos a la izquierda de la posición I+1 están **ordenados y en su posición definitiva**. Cada iteración hace **un intercambio**, aunque sea de un elemento consigo mismo (cuando K = I).

{ } Pseudocódigo

```
MÓDULO ordenacion_seleccion_directa
DATOS
   PARÁMETROS
      Transforma V() de tipo T
      Recibe N entera
   VARIABLES
      I, J, K enteras
      AUX de tipo T
INICIO
   Para I desde 1 hasta N-1
      K ← I
      AUX ← V(I)
      Para J desde I+1 hasta N
         Si ( V(J) < AUX ) entonces
            K ← J
            AUX ← V(J)
         fin-si
      fin-para
      V(K) ← V(I)
      V(I) ← AUX
   fin-para
FIN
```

ORDENACIÓN · 4

Intercambio directo

También llamado **método de la burbuja**

Se recorre el vector varias veces **comparando elementos consecutivos** e **intercambiándolos** si están mal colocados.

Izquierda → derecha

Los valores **mayores** se desplazan hacia la **derecha**. Cada pasada deja uno en su sitio en el **extremo derecho** de la zona.

Derecha → izquierda

Los valores **menores** se desplazan hacia la **izquierda**. Cada pasada deja uno en su sitio en el **extremo izquierdo** de la zona.

Cada pasada deja un elemento en su **posición definitiva** en el extremo de la zona comparada, así que en la siguiente pasada ese elemento **se excluye**. Se hacen **N−1 iteraciones**.

EJEMPLO

Versión izquierda → derecha

índice	1	2	3	4	5	6	7	8	
inicio	15	18	7	75	14	13	43	40	
J = 1	15	7	18	14	13	43	40	**75**	5 intercambios
J = 2	7	15	14	13	18	40	**43**	**75**	4 intercambios
J = 3	7	14	13	15	18	**40**	**43**	**75**	2 intercambios
J = 4	7	13	14	15	**18**	**40**	**43**	**75**	1 intercambio
J = 5	7	13	14	**15**	**18**	**40**	**43**	**75**	0 intercambios
J = 6	7	13	**14**	**15**	**18**	**40**	**43**	**75**	0 intercambios
J = 7	7	**13**	**14**	**15**	**18**	**40**	**43**	**75**	0 intercambios

▢ posición definitiva (extremo derecho)

EJEMPLO

Versión derecha → izquierda

índice	1	2	3	4	5	6	7	8	
inicio	15	18	7	75	14	13	43	40	
J = 1	7	15	18	13	75	14	40	43	el menor queda en V(1)
J = 2	7	13	15	18	14	75	40	43	el menor queda en V(2)
J = 3	7	13	14	15	18	40	75	43	el menor queda en V(3)
J = 4	7	13	14	15	18	40	43	75	el menor queda en V(4)
J = 5	7	13	14	15	18	40	43	75	el menor queda en V(5)
J = 6	7	13	14	15	18	40	43	75	el menor queda en V(6)
J = 7	7	13	14	15	18	40	43	75	el menor queda en V(7)

posición definitiva (extremo izquierdo)

Mejora: burbuja con test

Si en una pasada **no se produce ningún intercambio**, los datos ya están ordenados y se puede parar. Hace falta una **pasada completa** sin intercambios para poder hacerlo.

💡 Con esta mejora, el número de iteraciones depende del **grado de desorden inicial**. Con el vector de ejemplo hace 5 pasadas en lugar de 7, y con un vector ya ordenado solo 1.

{ } Pseudocódigo · izquierda → derecha

```
MÓDULO ordenacion_intercambio_directo_izq_der
DATOS
   PARÁMETROS
      Transforma V() de tipo T
      Recibe N entera
   VARIABLES
      I, J enteras
      AUX de tipo T
INICIO
   Para J desde 1 hasta N-1
      Para I desde 1 hasta N-J
         Si ( V(I) > V(I+1) ) entonces
            AUX ← V(I)
            V(I) ← V(I+1)
            V(I+1) ← AUX
         fin-si
      fin-para
   fin-para
FIN
```

{ } Pseudocódigo · derecha → izquierda

```
MÓDULO ordenacion_intercambio_directo_der_izq
DATOS
   PARÁMETROS
      Transforma V() de tipo T
      Recibe N entera
   VARIABLES
      I, J enteras
      AUX de tipo T
INICIO
   Para J desde 1 hasta N-1
      Para I desde N hasta J+1 con incremento -1
         Si ( V(I-1) > V(I) ) entonces
            AUX ← V(I-1)
            V(I-1) ← V(I)
            V(I) ← AUX
         fin-si
      fin-para
   fin-para
FIN
```

{ } Pseudocódigo · con test

```
MÓDULO ordenacion_intercambio_directo_test
DATOS
   PARÁMETROS
      Transforma V() de tipo T
      Recibe N entera
   VARIABLES
      I, J enteras
      SEGUIR lógica
      AUX de tipo T
INICIO
   J ← 1
   SEGUIR ← VERDADERO
   Mientras ( (SEGUIR = VERDADERO) y (J < N) ) hacer
      SEGUIR ← FALSO
      Para I desde 1 hasta N-J
         Si ( V(I) > V(I+1) ) entonces
            AUX ← V(I)
            V(I) ← V(I+1)
            V(I+1) ← AUX
            SEGUIR ← VERDADERO
         fin-si
      fin-para
      J ← J+1
   fin-mientras
FIN
```

ORDENACIÓN · 5

Sacudida

También llamado **ordenación por vibración**

Es una **mejora de la burbuja** que alterna sus dos versiones: una pasada de **izquierda a derecha** y después otra de **derecha a izquierda**. La zona a tratar se **recorta por ambos extremos**.

índice	1	2	3	4	5	6	7	8	
inicio	15	18	7	75	14	13	43	40	
→ izq-der	15	7	18	14	13	43	40	75	DER ← 7
← der-izq	7	15	13	18	14	40	43	75	IZQ ← 2
→ izq-der	7	13	15	14	18	40	43	75	DER ← 6
← der-izq	7	13	14	15	18	40	43	75	IZQ ← 3
→ izq-der	7	13	14	15	18	40	43	75	DER ← 5
← der-izq	7	13	14	15	18	40	43	75	IZQ ← 4
→ izq-der	7	13	14	15	18	40	43	75	DER ← 4

posiciones descartadas por los extremos

Sacudida con test

Igual que en la burbuja, se puede parar si en una pasada, ya sea de izquierda a derecha o de derecha a izquierda, se comprueba que todos los elementos están ordenados, es decir, que no hubo intercambios.

Sacudida con salto

Descarta **varias posiciones** de los extremos de una vez, en lugar de solo una:

- Pasada **derecha → izquierda**: si el último intercambio afecta a X y X+1, las posiciones **1..X** ya están ordenadas.
- Pasada **izquierda → derecha**: si el último intercambio afecta a X−1 y X, las posiciones **X..N** ya están ordenadas.

índice	1	2	3	4	5	6	7	8	
inicio	15	18	7	75	14	13	43	40	
→ izq-der	15	7	18	14	13	43	40	75	último intercambio en 7,8 → DER ← 7
← der-izq	7	15	13	18	14	40	43	75	último intercambio en 1,2 → IZQ ← 2
→ izq-der	7	13	15	14	18	40	43	75	último intercambio en 4,5 → DER ← 4
← der-izq	7	13	14	15	18	40	43	75	último intercambio en 3,4 → IZQ ← 4

posiciones descartadas (pueden ser varias de golpe)

Compara las dos trazas: la sacudida básica necesita 7 pasadas y la versión con salto termina en 4.

{ } Pseudocódigo · sacudida

```
MÓDULO ordenacion_sacudida
DATOS
   PARÁMETROS
      Transforma V() de tipo T
      Recibe N entera
   VARIABLES
      I, IZQ, DER enteras
      AUX de tipo T
INICIO
   IZQ ← 1
   DER ← N
   Mientras ( IZQ ≠ DER ) hacer
      ** pasada de izquierda a derecha
      Para I desde IZQ hasta DER-1
         Si ( V(I) > V(I+1) ) entonces
            AUX ← V(I)
            V(I) ← V(I+1)
            V(I+1) ← AUX
         fin-si
      fin-para
      DER ← DER-1

      Si ( IZQ ≠ DER ) entonces
         ** pasada de derecha a izquierda
         Para I desde DER hasta IZQ+1 con incremento -1
            Si ( V(I-1) > V(I) ) entonces
               AUX ← V(I-1)
               V(I-1) ← V(I)
               V(I) ← AUX
            fin-si
         fin-para
         IZQ ← IZQ+1
      fin-si
   fin-mientras
FIN
```

{ } **Pseudocódigo · sacudida con test**

```
MÓDULO ordenacion_sacudida_test
DATOS
   PARÁMETROS
      Transforma V() de tipo T
      Recibe N entera
   VARIABLES
      I, IZQ, DER, CEN enteras
      SEGUIR lógica
INICIO
   IZQ ← 1
   DER ← N
   SEGUIR ← VERDADERO
   Mientras ( (IZQ ≠ DER) y (SEGUIR = VERDADERO) ) hacer
      SEGUIR ← FALSO
      Para I desde IZQ hasta DER-1
         Si ( V(I) > V(I+1) ) entonces
            AUX ← V(I)
            V(I) ← V(I+1)
            V(I+1) ← AUX
            SEGUIR ← VERDADERO
         fin-si
      fin-para
      DER ← DER-1

      Si ( (IZQ ≠ DER) y (SEGUIR = VERDADERO) ) entonces
         SEGUIR ← FALSO
         Para I desde DER hasta IZQ+1 con incremento -1
            Si ( V(I-1) > V(I) ) entonces
               AUX ← V(I-1)
               V(I-1) ← V(I)
               V(I) ← AUX
               SEGUIR ← VERDADERO
            fin-si
         fin-para
         IZQ ← IZQ+1
      fin-si
   fin-mientras
FIN
```

{ } Pseudocódigo · sacudida con salto

```
MÓDULO ordenacion_sacudida_salto
DATOS
   PARÁMETROS
      Transforma V() de tipo T
      Recibe N entera
   VARIABLES
      I, IZQ, DER, K enteras
      AUX de tipo T
INICIO
   IZQ ← 1
   DER ← N
   K ← 1
   Mientras ( IZQ < DER ) hacer
      Para I desde IZQ hasta DER-1
         Si ( V(I) > V(I+1) ) entonces
            AUX ← V(I)
            V(I) ← V(I+1)
            V(I+1) ← AUX
            K ← I+1
         fin-si
      fin-para
      DER ← K-1

      Si ( IZQ < DER ) entonces
         Para I desde DER hasta IZQ+1 con incremento -1
            Si ( V(I-1) > V(I) ) entonces
               AUX ← V(I-1)
               V(I-1) ← V(I)
               V(I) ← AUX
               K ← I-1
            fin-si
         fin-para
         IZQ ← K+1
      fin-si
   fin-mientras
FIN
```

ORDENACIÓN · 6

Shell

También llamado **inserción con incrementos decrecientes**

Es una **mejora de la inserción directa**. Primero ordena por inserción los elementos separados **S1** posiciones, después los separados **S2 < S1**, y así sucesivamente hasta llegar a distancia **1**. Los valores S1, S2... se llaman **incrementos**.

1. Tomar una distancia de comparación inicial.
2. Ordenar entre sí los elementos situados a esa distancia.
3. Reducir la distancia. Si sigue siendo ≥ 1, volver al paso 2.

💡 En los apuntes, la distancia inicial es **la mitad del número de elementos** y en cada iteración se divide entre 2 (división entera), hasta llegar a **1**. Con N = 8 las distancias son **4 → 2 → 1**.

índice	1	2	3	4	5	6	7	8	
inicio	15	18	7	75	14	13	43	40	
distancia 4	14	13	7	40	15	18	43	75	ordenados entre sí los elementos a distancia 4
distancia 2	7	13	14	18	15	40	43	75	ordenados entre sí los elementos a distancia 2
distancia 1	7	13	14	15	18	40	43	75	inserción directa final → ordenado

mismo color = mismo subgrupo a esa distancia ordenado

⚠️ **Ojo:** la última pasada, con distancia 1, es una inserción directa normal y es la que **garantiza** que el vector quede ordenado. En el peor caso, toda la ordenación se hace en esa última pasada.

Pseudocódigo

```
MÓDULO ordenacion_shell
DATOS
   PARÁMETROS
      Transforma V() de tipo T
      Recibe N entera
   VARIABLES
      I, J, DISTANCIA, AUX enteras
INICIO
   DISTANCIA ← N
   Mientras ( DISTANCIA ≠ 1 ) hacer
      DISTANCIA ← DISTANCIA\2      ** división entera
      Para I desde DISTANCIA+1 hasta N
         AUX ← V(I)
         J ← I
         Mientras ( ((J-DISTANCIA) ≥ 1) y
                    (V(J-DISTANCIA) > AUX) ) hacer
            V(J) ← V(J-DISTANCIA)
            J ← J-DISTANCIA
         fin-mientras
         V(J) ← AUX
      fin-para
   fin-mientras
FIN
```

ORDENACIÓN · 7

QuickSort

Ordenación rápida · técnica **"divide y vencerás"**

Se elige un valor llamado **pivote**. Los elementos **menores** se llevan a su izquierda y los **mayores** a su derecha. Después se aplica lo mismo a cada una de las dos partes, de forma **recursiva**, hasta que todas las particiones tengan tamaño 1.

1. Se elige un elemento como pivote.
2. Si el vector tiene tamaño 1, ya está ordenado. Si no, se divide en **V1** y **V2**, de forma que los elementos de V1 sean menores (o iguales, si hay repetidos) que los de V2.
3. Se ordenan V1 y V2 aplicando QuickSort.
4. Se concatenan V1 y V2 y se obtiene V ordenado.

💡 **El pivote:** en principio puede ser **cualquier valor**, pero una mala elección **reduce mucho la eficiencia**. En los apuntes se toma el valor de la **posición central** del vector o subvector: `V((IZQ+DER)\2)` .

índice	1	2	3	4	5	6	7	8	
inicio	15	18	7	75	14	13	43	40	
(1,8) piv=75	15	18	7	40	14	13	43	75	pivote = V(4); izquierda [1..7] · derecha [8..8]
(1,7) piv=40	15	18	7	13	14	40	43	75	pivote = V(4); izquierda [1..5] · derecha [6..7]
(1,5) piv=7	7	18	15	13	14	40	43	75	pivote = V(3); izquierda [1..1] · derecha [2..5]
(2,5) piv=15	7	14	13	15	18	40	43	75	pivote = V(3); izquierda [2..3] · derecha [4..5]
(2,3) piv=14	7	13	14	15	18	40	43	75	pivote = V(2); izquierda [2..2] · derecha [3..3]
(4,5) piv=15	7	13	14	15	18	40	43	75	pivote = V(4); izquierda vacía · derecha [5..5]
(6,7) piv=40	7	13	14	15	18	40	43	75	pivote = V(6); izquierda vacía · derecha [7..7]

pivote · parte ≤ pivote · parte ≥ pivote · fuera de esta llamada

⚠️ **Ojo:** las dos partes **no tienen por qué ser del mismo tamaño**. En la primera llamada del ejemplo el pivote es 75, y las partes quedan de 7 elementos y de 1. Lo que se garantiza es que a la izquierda queda todo lo **≤ pivote** y a la derecha todo lo **≥ pivote**.

Pseudocódigo

```
MÓDULO ordenacion_rapida
DATOS
   PARÁMETROS
      Transforma V() de tipo T
      Recibe IZQ entera
      Recibe DER entera
   VARIABLES
      I, J enteras
      PIVOTE, AUX de tipo T
INICIO
   I ← IZQ
   J ← DER
   PIVOTE ← V((IZQ+DER)\2)   ** división entera

   Mientras ( I ≤ J ) hacer
      Mientras ( V(I) < PIVOTE ) hacer
         I ← I+1
      fin-mientras
      Mientras ( V(J) > PIVOTE ) hacer
         J ← J-1
      fin-mientras
      Si ( I ≤ J ) entonces
         AUX ← V(I)
         V(I) ← V(J)
         V(J) ← AUX
         I ← I+1
         J ← J-1
      fin-si
   fin-mientras

   Si ( IZQ < J ) entonces
      ordenacion_rapida(V, IZQ, J)
   fin-si
   Si ( DER > I ) entonces
      ordenacion_rapida(V, I, DER)
   fin-si
FIN
```

ORDENACIÓN · 8

MergeSort

Ordenación por mezcla · técnica **"divide y vencerás"**

1. Se **divide** el vector en dos mitades.
2. Se **ordena** cada mitad aplicando MergeSort.
3. Se **mezclan** las dos mitades y se obtiene un vector ordenado.

La división se aplica recursivamente. Una partición de **un solo elemento** se considera ordenada. Después se mezclan particiones de 1 para formar particiones ordenadas de 2, luego de 4... hasta tener todo el vector ordenado.

La operación de mezcla

Une dos vectores **ordenados** en uno nuevo que contiene todos sus elementos y **conserva el orden**. Aquí los dos "vectores" son en realidad dos tramos del vector original: **IZQ..CEN** y **CEN+1..DER**.

- Se compara un elemento de cada tramo, empezando por los primeros, y se copia el **menor** en el destino.
- Se avanza en el tramo del que se acaba de copiar y se sigue comparando.
- Cuando se han copiado **todos los elementos de un tramo**, se copian los que quedan del otro.
- El destino es un vector auxiliar **TEMP**. Al acabar, se copian en V las posiciones **IZQ..DER** de TEMP.

índice	1	2	3	4	5	6	7	8	
inicio	15	18	7	75	14	13	43	40	
mezclar(1,1,2)	**15**	**18**	7	75	14	13	43	40	[1..1] + [2..2]
mezclar(3,3,4)	15	18	**7**	**75**	14	13	43	40	[3..3] + [4..4]
mezclar(1,2,4)	**7**	**15**	**18**	**75**	14	13	43	40	[1..2] + [3..4]
mezclar(5,5,6)	7	15	18	75	**13**	**14**	43	40	[5..5] + [6..6]
mezclar(7,7,8)	7	15	18	75	13	14	**40**	**43**	[7..7] + [8..8]
mezclar(5,6,8)	7	15	18	75	**13**	**14**	**40**	**43**	[5..6] + [7..8]
mezclar(1,4,8)	**7**	**13**	**14**	**15**	**18**	**40**	**43**	**75**	[1..4] + [5..8]

tramo recién mezclado (ordenado)

💡 La **primera mezcla** que se hace une las posiciones **1 y 2**, dos elementos consecutivos, porque la recursión baja primero por la mitad izquierda.

{ } **Pseudocódigo**

```
MÓDULO ordenacion_por_mezcla
DATOS
   PARÁMETROS
      Transforma V() de tipo T
      Recibe IZQ entera
      Recibe DER entera
   VARIABLES
      CEN entera
INICIO
   Si ( IZQ < DER ) entonces
      CEN ← (IZQ+DER)\2      ** división entera
      ordenacion_por_mezcla(V, IZQ, CEN)
      ordenacion_por_mezcla(V, CEN+1, DER)
      mezclar(V, IZQ, CEN, DER)
   fin-si
FIN

MÓDULO mezclar
DATOS
   PARÁMETROS
      Transforma V() de tipo T
      Recibe IZQ entera
      Recibe CEN entera
      Recibe DER entera
   VARIABLES
      TEMP() de tipo T
      I1, I2, I_TEMP entera
INICIO
   I1 ← IZQ
   I2 ← CEN+1
   I_TEMP ← IZQ
   Mientras ( (I1 ≤ CEN) y (I2 ≤ DER) ) hacer
      Si ( V(I1) < V(I2) ) entonces
         TEMP(I_TEMP) ← V(I1)
         I1 ← I1 + 1
      sino
         TEMP(I_TEMP) ← V(I2)
         I2 ← I2 + 1
      fin-si
      I_TEMP ← I_TEMP + 1
   fin-mientras

   Mientras ( I1 ≤ CEN ) hacer
      TEMP(I_TEMP) ← V(I1)
      I1 ← I1 + 1
      I_TEMP ← I_TEMP + 1
   fin-mientras

   Mientras ( I2 ≤ DER ) hacer
      TEMP(I_TEMP) ← V(I2)
      I2 ← I2 + 1
      I_TEMP ← I_TEMP + 1
   fin-mientras

   Para I_TEMP desde IZQ hasta DER
      V(I_TEMP) ← TEMP(I_TEMP)
   fin-para
FIN
```

Ficheros secuenciales

La búsqueda y ordenación **externa** trabaja con datos guardados en **ficheros**, en memoria secundaria (disco), en lugar de en vectores en memoria principal.

Fichero, registro, campo y clave

Fichero o archivo: conjunto de informaciones estructuradas en unidades de acceso llamadas **registros**, todos del **mismo tipo**, guardadas en algún soporte de **memoria secundaria**.

TÉRMINO	SIGNIFICADO
Registro	Estructura formada por uno o más **campos**, que pueden ser de **tipos diferentes** y tener a su vez **subcampos**.
Clave (identificativo)	Campo especial que identifica al registro o sirve de criterio para las operaciones. Puede ser **única** o **repetirse**. Si hay varias, se habla de clave **primaria**, **secundaria**...
Acceso	Procedimiento para situarse sobre un registro concreto y leerlo o escribirlo.

💡 **Acceso secuencial:** los registros se recorren en el orden en que están guardados, de uno en uno desde el primero. Para llegar al registro **n** hay que pasar por los **n−1 anteriores**. No se puede saltar directamente al registro central, así que no se puede aplicar búsqueda binaria.

Operaciones básicas

Son cinco: **apertura**, **cierre**, **lectura**, **escritura** y **comprobación de fin de fichero**. La primera siempre es la apertura y la última el cierre.

Nombre físico

El del archivo en disco (`NOMBRE_F` , una cadena que puede incluir la ruta).

Nombre lógico

El que usa el programa (`F`). Se asocia al físico en la **apertura**, y desde ahí todas las operaciones usan el nombre lógico.

Para leer y escribir se usa una variable del mismo tipo que los registros, el **registro buffer** (`REG_BUFFER`), que intercambia registros entre el fichero y el programa.

Modos de apertura

MODO	QUÉ HACE	SI EL FICHERO NO EXISTE
Lectura	Para leer. El puntero de lectura queda sobre el **primer registro**.	Error en la apertura
Escritura	Para escribir **borrando** lo que hubiera. El puntero se coloca al **comienzo**.	Se crea
Añadir	Para escribir **conservando** el contenido. Se escribe a partir del **final**.	—

```
F ← Abrir NOMBRE_F para lectura
F ← Abrir NOMBRE_F para escritura
F ← Abrir NOMBRE_F para añadir
```

1. **Leer F, REG_BUFFER**: copia en el buffer el registro al que apunta el puntero de lectura y avanza el puntero al siguiente.
2. **Escribir F, REG_BUFFER**: coloca el contenido del buffer donde indica el puntero de escritura y lo avanza.
3. **FF(F)**: función lógica que vale **Verdadero** si el puntero de lectura está en la **marca de fin de fichero**, es decir, si no quedan registros por leer.
4. **Cerrar F**: libera el fichero para otros programas. Si estaba abierto para **escritura**, el cierre escribe la **marca de fin de fichero**.

⚠️ **Ojo:** antes de cada lectura hay que comprobar `FF(F)`. Leer cuando ya no quedan registros provoca un error.

TEMA 2 · ORDENACIÓN EXTERNA

Ordenación externa

Un fichero está **clasificado** (ordenado) de forma ascendente o descendente si todos sus registros están en ese orden según un campo, la **clave de ordenación**.

> 💡 **Primera pregunta:** si el fichero **cabe en memoria principal**, lo mejor es cargarlo en un vector, aplicar un método de **ordenación interna** y copiar los datos ordenados al fichero. Solo si no cabe se usa ordenación externa.

Los métodos de ordenación externa se basan en **particiones** y **mezclas** (fusiones):

Partición

Genera **dos o más ficheros** a partir de uno.

Mezcla o fusión

Combina ficheros ordenados, o parcialmente ordenados, en uno que **mantiene el orden**.

Ambas operaciones necesitan **ficheros auxiliares** y muchos **accesos a disco**. **Cuantos más accesos a disco, más lento** es el método.

Los ejemplos usan el fichero de los apuntes, con estas claves: **15 18 7 75 14 13 43 40 51 93 75 26 64 27 13**. Cada número representa la clave del registro; al mover la clave se mueve el registro entero.

ORDENACIÓN EXTERNA · 1

Mezcla directa

Hace particiones y mezclas sucesivas con secuencias de longitud **fija**, que se **duplica** en cada iteración: **1, 2, 4, 8...**

1. **Separar:** copia alternativamente en **F1** y en **F2** secuencias de `TAM_SEC` registros consecutivos de F. La última puede tener menos.
2. **Mezclar:** fusiona sobre F cada secuencia de F1 con la correspondiente de F2, y se obtienen secuencias ordenadas del doble de largo.
3. `TAM_SEC ← 2 * TAM_SEC` y se repite **hasta que TAM_SEC ≥ NUM_ELEM**, el número de registros.

> **Copia de cabos:** en cada mezcla, cuando ya no quedan registros de la secuencia de uno de los ficheros auxiliares, se copian en el destino los registros que quedan del otro.

F inicial 15 18 7 75 14 13 43 40 51 93 75 26 64 27 13

Iteración 1 · secuencias de longitud 1

F1 15 7 14 43 51 75 64 13

partición: tramos de 1

F2 18 75 13 40 93 26 27

F 15 18 7 75 13 14 40 43 51 93 26 75 27 64 13

fusión → secuencias ordenadas de 2

Iteración 2 · secuencias de longitud 2

F1 15 18 13 14 51 93 27 64

partición: tramos de 2

F2 7 75 40 43 26 75 13

F 7 15 18 75 13 14 40 43 26 51 75 93 13 27 64

fusión → secuencias ordenadas de 4

Iteración 3 · secuencias de longitud 4

F1 7 15 18 75 26 51 75 93

partición: tramos de 4

F2 13 14 40 43 13 27 64

F 7 13 14 15 18 40 43 75 13 26 27 51 64 75 93

fusión → secuencias ordenadas de 8

Iteración 4 · secuencias de longitud 8

F1 7 13 14 15 18 40 43 75

partición: tramos de 8

F2 13 26 27 51 64 75 93

F 7 13 13 14 15 18 26 27 40 43 51 64 75 75 93

fusión → fichero ordenado

los colores alternos marcan cada secuencia

⚠️ **Ojo en el examen:** el número de iteraciones depende solo del **número de registros** (con 15 registros: 1, 2, 4, 8, es decir, 4 iteraciones). **No** aprovecha el orden que ya tengan los datos: aunque el fichero esté ordenado, hace las mismas iteraciones.

Pseudocódigo · módulo principal

```
Módulo Clasificacion_mezcla_directa
DATOS
   PARÁMETROS
      Recibe NOMBRE_F cadena      ** fichero a ordenar
      Recibe NOMBRE_F1 cadena     ** primer fichero auxiliar
      Recibe NOMBRE_F2 cadena     ** segundo fichero auxiliar
   VARIABLES
      NUM_ELEM entero             ** num. de registros a ordenar
      TAM_SEC entero              ** tamaño de la secuencia actual
INICIO
   TAM_SEC ← 1
   Repetir
      Separar (NOMBRE_F, NOMBRE_F1, NOMBRE_F2, TAM_SEC, NUM_ELEM)
      Mezcla_directa (NOMBRE_F, NOMBRE_F1, NOMBRE_F2, TAM_SEC, NUM_ELEM)
      TAM_SEC ← 2 * TAM_SEC
   Hasta que (TAM_SEC ≥ NUM_ELEM)
FIN
```

{ } Pseudocódigo · Separar

```
Módulo Separar
DATOS
   PARÁMETROS
      Recibe NOMBRE_F cadena      ** fichero a ordenar
      Recibe NOMBRE_F1 cadena     ** primer fichero auxiliar
      Recibe NOMBRE_F2 cadena     ** segundo fichero auxiliar
      Recibe TAM_SEC entero       ** tamaño de la secuencia
      Transforma ELEM_F entero    ** registros del fichero origen
   VARIABLES
      REG de tipo REGISTRO
      F, F1, F2 archivos de REGISTRO
      N entera      ** número de reg. escritos en un destino
INICIO
   F1 ← Abrir NOMBRE_F1 para Escritura
   F2 ← Abrir NOMBRE_F2 para Escritura
   F ← Abrir NOMBRE_F para Lectura
   ELEM_F ← 0
   Leer F, REG
   ELEM_F ← ELEM_F + 1

   Mientras (FF(F) = Falso) hacer
      N ← 0
      Mientras ( (FF(F) = Falso) Y (N < TAM_SEC) ) hacer
         Escribir F1, REG
         N ← N+1
         Leer F, REG
         ELEM_F ← ELEM_F + 1
      fin-mientras
      N ← 0
      Mientras ( (FF(F) = Falso) Y (N < TAM_SEC) ) hacer
         Escribir F2, REG
         N ← N+1
         Leer F, REG
         ELEM_F ← ELEM_F + 1
      fin-mientras
   fin-mientras

   Cerrar F
   Cerrar F1
   Cerrar F2
FIN
```

{ } **Pseudocódigo · Mezcla_directa**

```
Módulo Mezcla_directa
DATOS
   PARÁMETROS
      Recibe NOMBRE_F cadena      ** fichero a ordenar
      Recibe NOMBRE_F1 cadena     ** primer fichero auxiliar
      Recibe NOMBRE_F2 cadena     ** segundo fichero auxiliar
      Recibe TAM_SEC entera       ** tamaño de la secuencia
      Recibe ELEM_F entera        ** registros del fichero a ordenar
   VARIABLES
      REG1, REG2 de tipo REGISTRO
      RESTA_F1 entero    ** reg. de la sec. del fich. 1 por mezclar
      RESTA_F2 entero    ** reg. de la sec. del fich. 2 por mezclar
      RESTA entero       ** reg. totales por mezclar
      F, F1, F2 archivos de REGISTRO
INICIO
   F1 ← Abrir NOMBRE_F1 para Lectura
   F2 ← Abrir NOMBRE_F2 para Lectura
   F ← Abrir NOMBRE_F para Escritura
   RESTA ← ELEM_F

   Repetir
      ** número de registros a mezclar del primer origen
      Si (RESTA ≥ TAM_SEC) entonces
         RESTA_F1 ← TAM_SEC
      sino
         RESTA_F1 ← RESTA
      fin-si
      RESTA ← RESTA - RESTA_F1

      ** número de registros a mezclar del segundo origen
      Si (RESTA ≥ TAM_SEC) entonces
         RESTA_F2 ← TAM_SEC
      sino
         RESTA_F2 ← RESTA
      fin-si
      RESTA ← RESTA - RESTA_F2

      Leer F1, REG1
      Leer F2, REG2
      Mientras ( (RESTA_F1 ≠ 0) Y (RESTA_F2 ≠ 0) ) hacer
         Si (REG1.CLAVE < REG2.CLAVE) entonces
            Escribir F, REG1
            RESTA_F1 ← RESTA_F1 - 1
            Si (RESTA_F1 ≠ 0) entonces
               Leer F1, REG1
            fin-si
         sino
            Escribir F, REG2
            RESTA_F2 ← RESTA_F2 - 1
            Si (RESTA_F2 ≠ 0) entonces
               Leer F2, REG2
            fin-si
         fin-si
      fin-mientras

      ** copia de cabos
      Mientras (RESTA_F2 ≠ 0) hacer
         Escribir F, REG2
         RESTA_F2 ← RESTA_F2 - 1
         Si (RESTA_F2 ≠ 0) entonces
            Leer F2, REG2
```

```
         fin-si
      fin-mientras
      Mientras (RESTA_F1 ≠ 0) hacer
         Escribir F, REG1
         RESTA_F1 ← RESTA_F1 - 1
         Si (RESTA_F1 ≠ 0) entonces
            Leer F1, REG1
         fin-si
      fin-mientras
   Hasta que (RESTA = 0)

   Cerrar F
   Cerrar F1
   Cerrar F2
FIN
```

ORDENACIÓN EXTERNA · 2

Mezcla natural

También hace particiones y mezclas, pero **aprovecha el orden que ya existe** en los datos: trabaja con **tramos ya ordenados** de longitud **variable**.

1. **Separar:** copia el primer tramo ordenado de F en F1, el segundo en F2, el tercero en F1... alternando hasta acabar el fichero.
2. **Mezclar:** fusiona sobre F cada tramo de F1 con un tramo de F2 y forma uno ordenado más largo. `L` cuenta cuántos tramos se han formado.
3. Se repite **hasta que L = 1**: si solo se ha formado un tramo, el fichero ya está ordenado.

Un tramo termina cuando se lee un registro cuya clave es **menor** que la del último copiado, o cuando se acaba el fichero (para orden ascendente). Si al mezclar uno de los ficheros auxiliares se acaba, los tramos que quedan en el otro se copian directamente: los apuntes lo llaman **copia de colas**.

F inicial 15 18 7 75 14 13 43 40 51 93 75 26 64 27 13
tramos ordenados del fichero original

Iteración 1

F1 15 18 14 40 51 93 26 64 13
partición: tramos ya ordenados, alternando

F2 7 75 13 43 75 27

F 7 15 18 75 13 14 40 43 51 75 93 26 27 64 13
fusión → L = 4 tramos

Iteración 2

F1 7 15 18 75 26 27 64
partición: tramos ya ordenados, alternando

F2 13 14 40 43 51 75 93 13

F 7 13 14 15 18 40 43 51 75 75 93 13 26 27 64
fusión → L = 2 tramos

Iteración 3

F1 7 13 14 15 18 40 43 51 75 75 93
partición: tramos ya ordenados, alternando

F2 13 26 27 64

F 7 13 13 14 15 18 26 27 40 43 51 64 75 75 93
fusión → L = 1 (un solo tramo: ordenado)

los colores alternos marcan cada tramo ordenado

⚠️ **Ojo:** al copiar tramos seguidos en un mismo fichero auxiliar, a veces se **juntan** y forman uno más largo. En la iteración 1 del ejemplo, el 14 y el 40 51 93 van seguidos en F1 y se unen en un solo tramo. Por eso los tramos que se mezclan no siempre coinciden con los de la partición.

💡 El número de iteraciones depende del **grado de desorden** de los datos, no solo del número de registros. Con este ejemplo hace 3 iteraciones frente a las 4 de la mezcla directa, y con un fichero **ya ordenado** haría solo **1**.

{ } **Pseudocódigo · módulo principal**

```
Módulo Clasificacion_mezcla_natural
DATOS
   PARÁMETROS
      Recibe NOMBRE_F cadena     ** fichero a ordenar
      Recibe NOMBRE_F1 cadena    ** primer fichero auxiliar
      Recibe NOMBRE_F2 cadena    ** segundo fichero auxiliar
   VARIABLES
      L entera      ** num. de subsecuencias ordenadas mezcladas
INICIO
   Repetir
      Separar (NOMBRE_F, NOMBRE_F1, NOMBRE_F2)
      Mezcla_natural (NOMBRE_F, NOMBRE_F1, NOMBRE_F2, L)
   Hasta que (L = 1)
FIN
```

{ } Pseudocódigo · Separar, Copiar_secuencia y Leer_y_chequear

```
Módulo Separar
DATOS
   PARÁMETROS
      Recibe NOMBRE_F cadena     ** fichero que se va a partir
      Recibe NOMBRE_F1 cadena    ** primer fichero auxiliar
      Recibe NOMBRE_F2 cadena    ** segundo fichero auxiliar
   VARIABLES
      REG de tipo REGISTRO
      F, F1, F2 archivos de REGISTRO
INICIO
   F1 ← Abrir NOMBRE_F1 para escritura
   F2 ← Abrir NOMBRE_F2 para escritura
   F ← Abrir NOMBRE_F para lectura
   Leer F, REG
   Repetir
      Copiar_secuencia (F, F1, REG)
      Si ( FF(F) = Falso ) entonces
         Copiar_secuencia (F, F2, REG)
      fin-si
   Hasta que (FF(F) = Verdadero)
   Cerrar F
   Cerrar F1
   Cerrar F2
FIN

Módulo Copiar_secuencia
DATOS
   PARÁMETROS
      Recibe F_O archivo de REGISTRO       ** fichero origen
      Transforma F_D archivo de REGISTRO   ** fichero destino
      Transforma REG de tipo REGISTRO      ** reg. leído del origen
   VARIABLES
      PARAR lógica
INICIO
   PARAR ← Falso
   Repetir
      Escribir F_D, REG
      Leer_y_chequear (F_O, REG, PARAR)
   Hasta (PARAR = Verdadero)
FIN

Módulo Leer_y_chequear
DATOS
   PARÁMETROS
      Recibe FICH archivo de REGISTRO   ** fichero abierto para lectura
      Transforma REG de tipo REGISTRO   ** reg. leído del origen
      Transforma PARAR lógica
   VARIABLES
      REG_SIG de tipo REGISTRO
INICIO
   Si (FF(FICH) = Falso) entonces
      Leer FICH, REG_SIG
      Si (REG.CLAVE > REG_SIG.CLAVE) entonces
         PARAR ← Verdadero
      fin-si
      REG ← REG_SIG
   sino
      PARAR ← Verdadero
   fin-si
FIN
```

{ } **Pseudocódigo · Mezcla_natural**

```
Módulo Mezcla_natural
DATOS
   PARÁMETROS
      Recibe NOMBRE_F cadena      ** fichero resultado
      Recibe NOMBRE_F1 cadena     ** primer fichero auxiliar
      Recibe NOMBRE_F2 cadena     ** segundo fichero auxiliar
      Transforma L entera         ** num. de secuencias mezcladas
   VARIABLES
      R1, R2 de tipo REGISTRO
      FIN_ORDEN booleana
      F, F1, F2 archivo de REGISTRO
INICIO
   F1 ← Abrir NOMBRE_F1 para lectura
   F2 ← Abrir NOMBRE_F2 para lectura
   F ← Abrir NOMBRE_F para escritura
   L ← 0
   Leer F1, R1
   Leer F2, R2

   Repetir
      FIN_ORDEN ← Falso
      Mientras (FIN_ORDEN = Falso) hacer
         Si (R1.CLAVE < R2.CLAVE) entonces
            Escribir F, R1
            Leer_y_chequear (F1, R1, FIN_ORDEN)
            Si (FIN_ORDEN = Verdadero) entonces
               Copiar_secuencia (F2, F, R2)
            fin-si
         sino
            Escribir F, R2
            Leer_y_chequear (F2, R2, FIN_ORDEN)
            Si (FIN_ORDEN = Verdadero) entonces
               Copiar_secuencia (F1, F, R1)
            fin-si
         fin-si
      fin-mientras
      L ← L+1
   Hasta que ( (FF(F1) = Verdadero) O (FF(F2) = Verdadero) )

   ** copias de colas
   Mientras (FF(F1) = Falso) hacer
      Copiar_secuencia (F1, F, R1)
      L ← L+1
   fin-mientras
   Mientras (FF(F2) = Falso) hacer
      Copiar_secuencia (F2, F, R2)
      L ← L+1
   fin-mientras
FIN
```

ORDENACIÓN EXTERNA · 3

Clasificación de raíz

Solo claves numéricas enteras

En cada iteración hace una **partición** y una **concatenación**, no una mezcla. Reparte los registros en **10 ficheros auxiliares** (F0...F9) según una cifra de la clave y después los junta en orden.

1. **Partición:** cada registro va al fichero de su cifra en la iteración actual. Si la cifra es 0 va a F0, si es 1 a F1... y si es 9 a F9.
2. **Concatenación:** se copian sobre el fichero original, por orden, F0, F1... F9.
3. Las cifras se toman de **derecha a izquierda**: primero las **unidades**, luego las decenas... Hay **tantas iteraciones como cifras tenga la clave más larga**.

Las claves con menos cifras se completan con ceros a la izquierda: en el ejemplo, el 7 es 07. Como la clave más larga tiene 2 cifras, hacen falta **2 iteraciones**.

F inicial	15 18 07 75 14 13 43 40 51 93 75 26 64 27 13

Iteración 1 · partición por la cifra de las unidades

F0	40
F1	51
F2	–
F3	13 43 93 13
F4	14 64
F5	15 75 75
F6	26
F7	07 27
F8	18
F9	–
F	40 51 13 43 93 13 14 64 15 75 75 26 07 27 18 concatenación F0 → F9

Iteración 2 · partición por la cifra de las decenas

F0	07
F1	13 13 14 15 18
F2	26 27
F3	–
F4	40 43
F5	51
F6	64
F7	75 75
F8	–
F9	93
F	07 13 13 14 15 18 26 27 40 43 51 64 75 75 93 concatenación F0 → F9

💡 `Obtener_cifra` saca la cifra con `CLAVE MOD 10` y `CLAVE DIV 10`, repetidos tantas veces como indique la iteración. Si después de quitar esa cifra todavía queda número (N ≠ 0), hay otra iteración: `SEGUIR ← Verdadero`.

{ } **Pseudocódigo · módulo principal**

```
Módulo Clasificacion_de_raiz
DATOS
   PARÁMETROS
      Recibe NOMBRE_F cadena                ** fichero original
      Recibe NOMBRE_F_AUX(10) de cadenas    ** ficheros auxiliares
   VARIABLES
      SEGUIR lógica       ** seguir/no seguir con las iteraciones
      ITERACION entera    ** contador de iteraciones
INICIO
   SEGUIR ← Verdadero
   ITERACION ← 1
   Repetir
      Separar (NOMBRE_F, NOMBRE_F_AUX, ITERACION, SEGUIR)
      Concatenar (NOMBRE_F, NOMBRE_F_AUX)
      ITERACION ← ITERACION + 1
   Hasta que (SEGUIR = Falso)
FIN
```

{ } Pseudocódigo · Separar, Obtener_cifra y Concatenar

```
Módulo Separar
DATOS
   PARÁMETROS
      Recibe NOMBRE_ORIGEN cadena            ** fichero a ordenar
      Recibe NOMBRE_DESTINO(10) de cadenas   ** ficheros auxiliares
      Recibe ITERACION entera                ** iteración actual
      Transforma SEGUIR lógica               ** seguir/no seguir ordenando
   VARIABLES
      REG de tipo REGISTRO
      CIFRA entero
      F_O fichero de REGISTRO               ** fichero origen
      F_D(10) de ficheros de REGISTRO       ** ficheros destino
      I entera
INICIO
   SEGUIR ← Falso
   Para I desde 1 hasta 10
      F_D(I) ← Abrir NOMBRE_DESTINO(I) para escritura
   fin-para
   F_O ← Abrir NOMBRE_ORIGEN para lectura

   Mientras ( FF(F_O) = Falso ) hacer
      Leer REG, F_O
      CIFRA ← Obtener_cifra(REG.CLAVE, ITERACION, SEGUIR)
      Escribir REG, F_D(CIFRA+1)
   fin-mientras

   Para I desde 1 hasta 10
      Cerrar F_D(I)
   fin-para
   Cerrar F_O
FIN

MÓDULO Obtener_cifra devuelve entero
DATOS
   PARÁMETROS
      Recibe CLAVE entero         ** clave de un registro
      Recibe ITERACION entero     ** iteración actual
      Transforma SEGUIR lógica    ** seguir/no seguir iterando
   VARIABLES
      I entero
      CIFRA entero
      N entero      ** copia de la clave
INICIO
   N ← CLAVE
   Para I desde 1 hasta ITERACION
      CIFRA ← N MOD 10    ** resto de la división
      N ← N DIV 10        ** división entera
   fin-para
   Si (N ≠ 0) entonces
      SEGUIR ← Verdadero
   fin-si
   DEVOLVER CIFRA
FIN

Módulo Concatenar
DATOS
   PARÁMETROS
      Recibe NOMBRE_DESTINO cadena           ** fichero a ordenar
      Recibe NOMBRE_ORIGEN(10) de cadenas    ** ficheros auxiliares
   VARIABLES
      REG de tipo REGISTRO
```

```
      F_D fichero de REGISTRO
      F_O(10) de ficheros de REGISTRO
      I entera
INICIO
   Para I desde 1 hasta 10
      F_O(I) ← Abrir NOMBRE_ORIGEN(I) para lectura
   fin-para
   F_D ← Abrir NOMBRE_DESTINO para escritura
   Para I desde 1 hasta 10
      Mientras ( FF(F_O(I)) = Falso ) hacer
         Leer REG, F_O(I)
         Escribir REG, F_D
      fin-mientras
   fin-para
   Para I desde 1 hasta 10
      Cerrar F_O(I)
   fin-para
   Cerrar F_D
FIN
```

TEMA 2 · BÚSQUEDA EXTERNA

Búsqueda en ficheros

Consiste en localizar el registro o los registros cuyo campo de búsqueda, la **clave de búsqueda**, tenga un valor dado. Si no se encuentra, hay que informar al usuario o al módulo que pidió la búsqueda.

Fichero que puede estar desordenado
buscar una coincidencia · buscar todas

Fichero ordenado por la clave
buscar una coincidencia · buscar todas

💡 Si el fichero está **ordenado** de forma ascendente por la clave de búsqueda, se puede **parar** al leer un registro con clave **mayor** que la buscada (**menor**, si el orden es descendente). Los algoritmos para ficheros desordenados también sirven para ficheros ordenados.

CASO	¿CUÁNDO TERMINA?
Una coincidencia, desordenado	Al encontrarlo (éxito) o al llegar al final (fracaso). Si hay repetidos, encuentra el **más próximo al comienzo**.
Todas las coincidencias, desordenado	Siempre al **final del fichero**. Aplica `TRATAR` a cada coincidencia; `ENCONTRADO` indica si hubo alguna.
Una coincidencia, ordenado	Al encontrarlo, al leer una clave **mayor** o al llegar al final.
Todas las coincidencias, ordenado	Al leer una clave **mayor** o al llegar al final.

⚠️ **Ojo:** en un fichero secuencial **no se puede aplicar búsqueda binaria**, aunque esté ordenado, porque no se puede ir directamente al registro central.

{ } Pseudocódigo · una coincidencia (desordenado)

```
Módulo Buscar_uno devuelve lógico
DATOS
   PARÁMETROS
      Recibe NOMBRE_F cadena    ** fichero a procesar
      Recibe X de tipo T        ** clave que se busca
      Transforma REG_BUSCADO de tipo REGISTRO
   VARIABLES
      ENCONTRADO lógico
      REG de tipo REGISTRO
      F archivo de REGISTRO
INICIO
   F ← Abrir NOMBRE_F para lectura
   ENCONTRADO ← Falso
   Mientras ( (ENCONTRADO = Falso) Y (FF(F) = Falso) ) hacer
      Leer F, REG
      Si (REG.CLAVE = X) entonces
         REG_BUSCADO ← REG
         ENCONTRADO ← Verdadero
      fin-si
   fin-mientras
   Cerrar F
   Devolver ENCONTRADO
FIN
```

{ } Pseudocódigo · todas las coincidencias (desordenado)

```
Módulo Buscar_todos
DATOS
   PARÁMETROS
      Recibe NOMBRE_F cadena    ** fichero a procesar
      Recibe X de tipo T        ** clave que se busca
   VARIABLES
      REG de tipo REGISTRO
      ENCONTRADO lógico
      F archivo de REGISTRO
INICIO
   F ← Abrir NOMBRE_F para lectura
   ENCONTRADO ← Falso
   Mientras ( FF(F) = Falso ) hacer
      Leer F, REG
      Si (REG.CLAVE = X) entonces
         TRATAR REG
         ENCONTRADO ← Verdadero
      fin-si
   fin-mientras
   Si (ENCONTRADO = Falso) entonces
      Escribir "Valor no encontrado"
   fin-si
   Cerrar F
FIN
```

{ } **Pseudocódigo · una coincidencia (ordenado)**

```
Módulo Buscar_uno_ordenado devuelve lógico
DATOS
   PARÁMETROS
      Recibe NOMBRE_F cadena     ** fichero a procesar
      Recibe X de tipo T         ** clave que se busca
      Transforma REG_BUSCADO de tipo REGISTRO
   VARIABLES
      F archivo de REGISTRO
      ACABAR lógico
      REG de tipo REGISTRO
INICIO
   F ← Abrir NOMBRE_F para lectura
   ACABAR ← Falso
   Mientras ( (ACABAR = Falso) Y (FF(F) = Falso) ) hacer
      Leer F, REG
      Si (REG.CLAVE ≥ X) entonces     ** en los apuntes aparece "⩽"
         ACABAR ← Verdadero
      fin-si
   fin-mientras
   Si (ACABAR = Verdadero) entonces
      Si (REG.CLAVE = X) entonces
         REG_BUSCADO ← REG
      sino
         ACABAR ← Falso
      fin-si
   fin-si
   Cerrar F
   Devolver ACABAR
FIN
```

En el PDF, la condición de `Buscar_uno_ordenado` aparece como `REG.CLAVE ⩽ X`. Parece una errata: así la búsqueda pararía en el primer registro con clave menor que X. Para que pare al llegar a la clave buscada o a una mayor, como explica el texto, debe ser `≥`. Consúltalo con tu profesor si te lo piden en el examen.

{ } Pseudocódigo · todas las coincidencias (ordenado)

```
Módulo Buscar_todos_ordenado
DATOS
   PARÁMETROS
      Recibe NOMBRE_F cadena     ** fichero a procesar
      Recibe X de tipo T         ** clave buscada
   VARIABLES
      SALIR, EXISTE lógicos
      REG de tipo REGISTRO
      F archivo de REGISTRO
INICIO
   SALIR ← Falso
   EXISTE ← Falso
   F ← Abrir NOMBRE_F para lectura
   Mientras ( (SALIR = Falso) Y (FF(F) = Falso) ) hacer
      Leer F, REG
      Si (REG.CLAVE > X) entonces
         SALIR ← Verdadero
      sino
         Si (REG.CLAVE = X) entonces
            TRATAR REG
            EXISTE ← Verdadero
         fin-si
      fin-si
   fin-mientras
   Si (EXISTE = Falso) entonces
      Escribir "No existen coincidencias"
   fin-si
   Cerrar F
FIN
```

REPASO

Resumen y comparativa

MÉTODO	OTRO NOMBRE	IDEA	¿DEJA UN ELEMENTO DEFINITIVO EN CADA ITERACIÓN?	SE BASA EN...
Inserción directa	Baraja	Inserta cada elemento entre los anteriores, que ya están ordenados	**No**	—
Inserción binaria	—	Igual, pero busca el hueco con búsqueda binaria	**No**	Inserción directa
Selección directa	—	Busca el menor de lo que falta y lo intercambia con V(I)	**Sí (N−1 iteraciones)**	—
Intercambio directo	Burbuja	Compara e intercambia elementos consecutivos	**Sí, en el extremo**	—
Sacudida	Vibración	Burbuja alternando sentidos y recortando por los dos extremos	**Sí, en ambos extremos**	Intercambio directo
Shell	Incrementos decrecientes	Inserción a distancias N/2, N/4... hasta 1	**No**	Inserción directa
QuickSort	Ordenación rápida	Parte el vector alrededor de un pivote y se aplica recursivamente	**No***	Divide y vencerás
MergeSort	Ordenación por mezcla	Divide en mitades, ordena cada una y las mezcla	**No**	Divide y vencerás

* En QuickSort no hay un "bucle externo" que fije un elemento por iteración: lo que se garantiza es la separación ≤ pivote / ≥ pivote.

Ordenación externa: comparativa

MÉTODO	FICHEROS AUXILIARES	OPERACIONES	SECUENCIAS	TERMINA CUANDO...	REQUISITO
Mezcla directa	2	Partición + mezcla	Longitud fija: 1, 2, 4, 8...	TAM_SEC ≥ nº de registros	—
Mezcla natural	2	Partición + mezcla	Tramos ya ordenados, de longitud variable	L = 1 (un solo tramo)	—
Raíz	10	Partición + **concatenación**	Una cifra de la clave por iteración, empezando por las unidades	Se procesan todas las cifras de la clave más larga	**Claves enteras**

¿De qué depende el número de iteraciones?

Siempre fijo (según N)

Selección directa (N−1), burbuja sin test (N−1), sacudida básica, Shell (una pasada por distancia).

Depende del desorden inicial

Burbuja con test, sacudida con test y sacudida con salto: paran antes si los datos ya están (casi) ordenados.

Trampas típicas de examen

Búsqueda binaria solo si el vector está ordenado **por el campo que se busca**. Un vector de alumnos ordenado por DNI no permite buscar una nota con búsqueda binaria: hay que usar la lineal.

Burbuja con test: hace falta una **pasada completa** sin intercambios. No basta con que una comparación no intercambie.

Inserción (directa o binaria): la zona izquierda está ordenada, pero **no es definitiva**.

QuickSort: no divide en mitades iguales; eso lo hace MergeSort. El pivote **sí** influye en la eficiencia.

Burbuja izquierda → derecha lleva los **mayores** hacia el final; la de derecha → izquierda lleva los **menores** hacia el principio.

Orden descendente: se invierte la comparación. En la burbuja se intercambia si la primera es **menor** que la segunda.

Búsqueda lineal: funciona con cualquier tipo de dato y con vectores ordenados o no. Si hay repetidos, encuentra el más cercano al punto de inicio.

Shell se basa en la inserción y la **sacudida** en la burbuja. ¡No los confundas!

Mezcla directa no aprovecha el orden previo: sus iteraciones dependen solo del **número de registros**. La **mezcla natural** sí lo aprovecha: dependen del **desorden**.

Raíz hace partición y **concatenación**, no mezcla, y solo sirve con **claves enteras**.

Ficheros secuenciales: nada de búsqueda binaria. Y si el fichero cabe en memoria, es mejor cargarlo y usar ordenación interna.

Mezcla natural en orden descendente: un tramo termina cuando se lee un valor **mayor** que el anterior, no menor.

REPASO FINAL

Tabla de métodos de ordenación

Todos los métodos del temario en dos tablas: la primera dice qué hace cada uno y con qué datos y estructuras funciona, y la segunda cuál es más rápido.

Qué hace cada método y dónde se puede usar

MÉTODO	QUÉ HACE (RESUMEN)	TIPO DE CLAVE	MEMORIA ESTÁTICA	MEMORIA DINÁMICA		FICHERO SECUENCIAL (DISCO)
			VECTOR NORMAL	VECTOR CON MALLOC	LISTA ENLAZADA	
ORDENACIÓN INTERNA · DATOS EN MEMORIA						
Inserción directa baraja	Inserta cada elemento entre los anteriores, que ya están ordenados, desplazando los mayores.	Cualquiera con orden	Sí	Sí	Sí	No
Inserción binaria	Igual, pero busca el hueco con búsqueda binaria.	Cualquiera con orden	Sí	Sí	No: la búsqueda binaria necesita ir directamente al elemento central	No
Selección directa	Busca el menor de lo que queda y lo intercambia con V(I).	Cualquiera con orden	Sí	Sí	Sí	No
Intercambio directo burbuja	Compara e intercambia elementos consecutivos. Cada pasada fija uno en el extremo.	Cualquiera con orden	Sí	Sí	Sí	No
Sacudida vibración	Burbuja alternando los dos sentidos y recortando por ambos extremos.	Cualquiera con orden	Sí	Sí	Solo doblemente enlazada (recorre hacia atrás)	No
Shell incrementos decrecientes	Inserción entre elementos a distancia N/2, N/4... hasta 1.	Cualquiera con orden	Sí	Sí	No: salta posiciones	No
QuickSort	Pone los menores a un lado del pivote y los mayores al otro, y repite en cada parte (recursivo).	Cualquiera con orden	Sí	Sí	No tal como está en los apuntes: pivote central y recorrido hacia atrás	No
MergeSort	Divide en mitades, ordena cada una y las mezcla (recursivo).	Cualquiera con orden	Sí (TEMP es otro vector)	Sí (TEMP se puede reservar con malloc)	Sí	No: su versión para ficheros es la mezcla directa
ORDENACIÓN EXTERNA · DATOS EN FICHEROS						
Mezcla directa	Parte en F1/F2 secuencias de 1, 2, 4, 8... y las mezcla. 2 ficheros auxiliares.	Cualquiera con orden	No: los datos no están en memoria, sino en ficheros en disco			Sí
Mezcla natural	Parte en F1/F2 los tramos ya ordenados y los mezcla hasta que queda uno. 2 ficheros auxiliares.	Cualquiera con orden	No: los datos no están en memoria, sino en ficheros en disco			Sí
Raíz	Reparte por cifras (unidades, decenas...) en F0...F9 y concatena. 10 ficheros auxiliares.	Solo enteros	No: los datos no están en memoria, sino en ficheros en disco			Sí

💡 **Resumiendo:** todos los métodos internos funcionan con **memoria estática** (un vector normal) y con **memoria dinámica** si es un vector creado con malloc, porque se usa igual, con índices. Con una **lista enlazada**, también dinámica, solo funcionan los que recorren los elementos de uno en uno: inserción directa, selección, burbuja, MergeSort y sacudida (esta solo si la lista es doble). Los métodos externos no usan ni una ni otra: trabajan con **ficheros en disco**.

"Cualquiera con orden": números, cadenas (por orden lexicográfico) o registros ordenados por uno de sus campos. Las columnas de memoria son mi análisis: los apuntes solo describen estos métodos sobre vectores y ficheros.

¿Cuál es más rápido?

Aviso: los apuntes no dan la complejidad (O grande) de cada método. Esta tabla es un **complemento** con los valores habituales; para el examen, lo importante son las ideas de las columnas de la derecha, que sí salen en los apuntes.

MÉTODO	MEJOR CASO	CASO MEDIO	PEOR CASO	MEMORIA EXTRA	¿APROVECHA EL ORDEN PREVIO?	¿ELEMENTO DEFINITIVO EN CADA ITERACIÓN?
ORDENACIÓN INTERNA						
Inserción directa	O(n)	$O(n^2)$	$O(n^2)$	Nada	Sí: si ya está ordenado, no desplaza nada	No
Inserción binaria	O(n log n)	$O(n^2)$	$O(n^2)$	Nada	Menos comparaciones, mismos desplazamientos	No
Selección directa	$O(n^2)$	$O(n^2)$	$O(n^2)$	Nada	No: siempre N−1 iteraciones	Sí
Burbuja	$O(n^2)$ · O(n) con test	$O(n^2)$	$O(n^2)$	Nada	Solo con test	Sí, en el extremo
Sacudida	O(n) con test o salto	$O(n^2)$	$O(n^2)$	Nada	Con test o salto	Sí, en ambos extremos
Shell	O(n log n)	$\approx O(n^{1,5})$	$O(n^2)$ con N/2, N/4...	Nada	En parte	No
QuickSort	O(n log n)	O(n log n)	$O(n^2)$ con mal pivote	Pila de la recursión	No	No
MergeSort	O(n log n)	O(n log n)	O(n log n)	Vector TEMP (n)	No	No
ORDENACIÓN EXTERNA (ITERACIONES = PASADAS COMPLETAS POR EL FICHERO)						
Mezcla directa	$\log_2$ n iteraciones	$\log_2$ n	$\log_2$ n	2 ficheros	No: depende solo del nº de registros	—
Mezcla natural	1 iteración (ya ordenado)	$\log_2$ (nº de tramos)	$\approx \log_2$ n	2 ficheros	Sí: depende del desorden	—
Raíz	Tantas iteraciones como cifras tenga la clave más larga			10 ficheros	No	—

En resumen

Datos desordenados y muchos elementos: lo más rápido es **QuickSort** (en la práctica, el más rápido) y **MergeSort** (siempre n log n, pero usa memoria extra). Les sigue **Shell**. Los demás son $O(n^2)$.

Datos casi ordenados: inserción directa, **burbuja con test** y **sacudida con test o salto** se acercan a O(n), porque aprovechan el orden.

Entre los $O(n^2)$: la **inserción** suele ser la más rápida, la **selección** hace menos intercambios (solo N−1) y la **burbuja sin test** es la más lenta.

Ficheros: si cabe en memoria, es mejor **cargarlo y usar ordenación interna**. Si no cabe, la **mezcla natural** aprovecha el orden, la **directa** siempre hace lo mismo y **raíz** solo sirve con claves enteras.